

Note méthodologique : preuve de concept

Dataset retenu

Pour nos données textuelles, le dataset choisi représente 1050 articles destinés à être vendu sur une plateforme de vente en ligne. Il se présente sous la forme d'une unique table contenant une ligne par article et fait apparaître en particulier les noms, les catégories et les descriptions des produits.

Le nom et la description d'un article sont uniques et deux articles ont toujours des noms et descriptions différents. Chaque article appartient à une unique catégorie (on peut ainsi parler de classe).

Les produits sont équi-répartis dans 7 catégories (→ classes équilibrés) :

Home Furnishing	150
Baby Care	150
Watches	150
Home Decor & Festive Needs	150
Kitchen & Dining	150
Beauty and Personal Care	150
Computers	150

Les noms d'articles sont plutôt courts, moins de 30 mots au maximum et moins de 10 mots en médiane.

Les descriptions quant à elles sont plus longues mais reste concises : un maximum proche des 600 mots mais une médiane inférieure à 50 mots.

Les concepts de l'algorithme récent

L'algorithme vise à effectuer une tâche de classification sur des données textuelles (dans l'application de cet article: déterminer si un document est pertinent pour une requête, le document et la requête jouent un rôle symétrique, et la classe est 0 ou 1 selon qu'un couple (document, requête) est pertinent ou non). Il combine deux représentations complémentaires du texte :

- TF-IDF, qui mesure l'importance des mots selon leur fréquence d'apparition ;
- Sentence-BERT, qui représente le sens global du document.

Deux encodages sont donc réalisés : un encodage TF-IDF suivi d'une réduction de dimension SVD, et un encodage par un Sentence-BERT (avec un modèle pré-entraîné : [sentence-transformers/all-MiniLM-L6-v2 · Hugging Face](#)). Ces deux encodages

(vecteurs de même dimensions grâce à la réduction de dimension effectuée) sont ensuite sommés de manière pondérés : un poids de 0,9 pour SBERT et 0,1 pour TF-IDF.

La phase de classification est ensuite effectuée par un réseau de neurones fully-connected :

- une couche dense de 256 neurones, avec activation ReLU
- une couche de dropout à 30 %, destinée à limiter le surapprentissage
- une couche dense de 64 neurones, avec activation ReLU
- une couche de sortie à un neurone, avec activation sigmoïde,

qui joue donc le rôle de classificateur binaire.

La modélisation

Contrairement à ce qui est fait dans l'article, et parce que le nature de notre jeu de données est un peu différente et que nos noms de catégories sont très courts, nous ne travaillons pas sur une classification binaire : nos features seront les noms/descriptions des produits et nos classes seront directement les catégories des produits.

En conséquence, nous travaillerons sur des vecteurs de features de taille 384 et non 768.

Tout comme dans l'article, nous faisons un preprocessing minimal de notre jeu de données :

- concaténation des noms et descriptions produit, pour ne pas perdre d'information en considérant seulement les descriptions.
- mise en minuscule et suppression de la ponctuation dans les descriptions.

Ensuite les données sont encodées via TF-IDF puis on applique une réduction de dimension SVD (encore une fois dimension 384 et non 768). Tout comme dans l'article, nous encodons également via un SBERT pré-entraîné (également [sentence-transformers/all-MiniLM-L6-v2 · Hugging Face](#)), et nous moyennons nos features en donnant les mêmes poids aux features TF-IDF et SBERT, à savoir 0,9 pour les features SBERT et 0,1 pour les features TF-IDF.

Le Fully-connected Neural Network (FCNN) employé est identique à celui détaillé dans l'article (mais encore une fois avec une dimension d'entrée de 384 et non 768). Etant donné que notre jeu de données est très réduit (seulement 1050 lignes), nous testons également en remplaçant le FCNN par un SVM plus simple, en optimisant via un gridsearchCV.

En ce qui concerne l'évaluation des résultats, étant donné que notre jeu de données est parfaitement équilibré et que nous cherchons simplement à maximiser le nombre

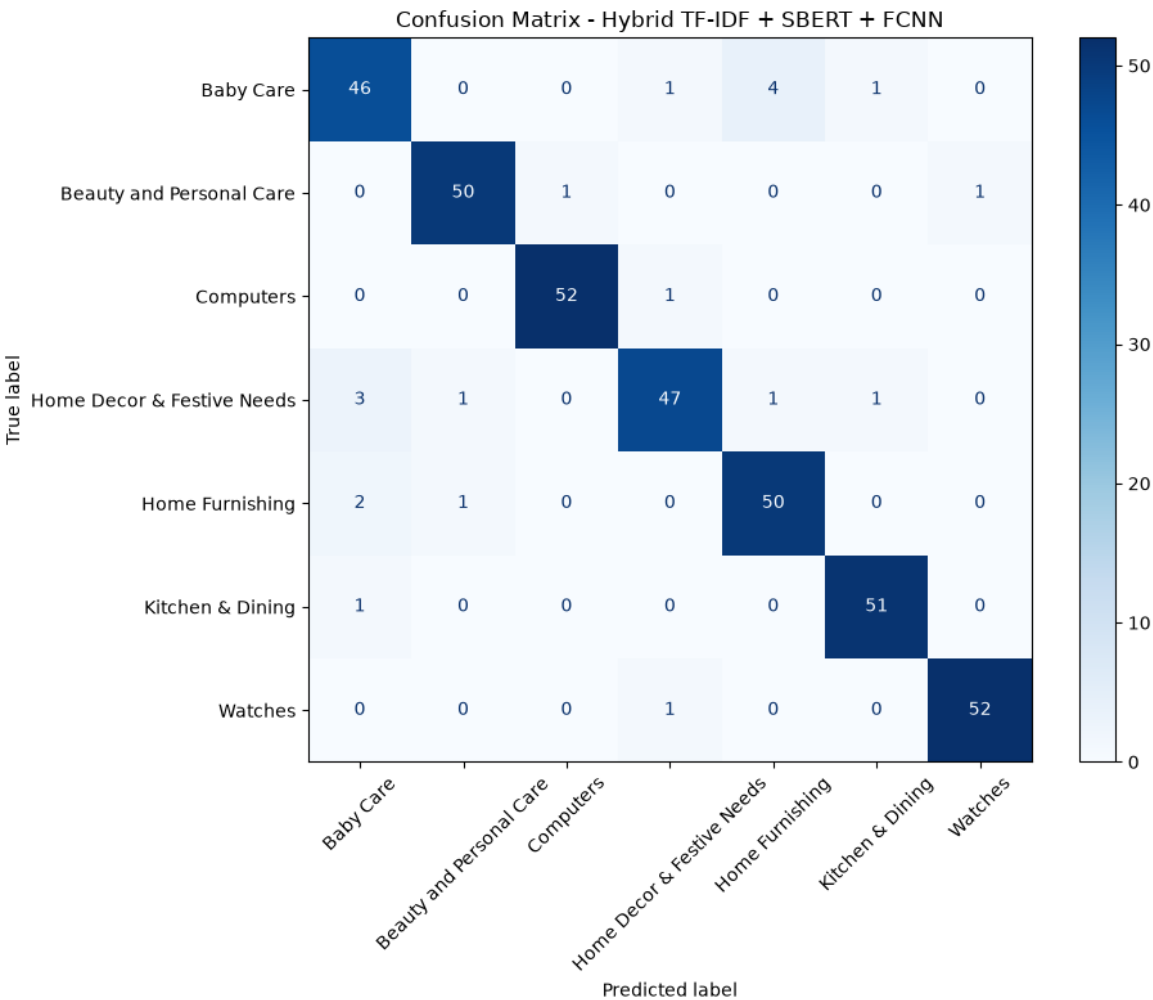
de classifications correctes, nous nous contenterons de regarder l'accuracy ($=\frac{\text{\#classification correcte}}{\text{\#classification}}$).

Une synthèse des résultats

=== RESULTS ===
Accuracy: 0.9457

Classification Report:

	precision	recall	f1-score	support
Baby Care	0.885	0.885	0.885	52
Beauty and Personal Care	0.962	0.962	0.962	52
Computers	0.981	0.981	0.981	53
Home Decor & Festive Needs	0.940	0.887	0.913	53
Home Furnishing	0.909	0.943	0.926	53
Kitchen & Dining	0.962	0.981	0.971	52
Watches	0.981	0.981	0.981	53
accuracy			0.946	368
macro avg	0.946	0.946	0.945	368
weighted avg	0.946	0.946	0.946	368



Les résultats obtenus sont meilleurs qu'avec des méthodes plus élémentaires, notamment qu'un simple BERT + SVM (accuracy de 0,913), cf paragraphes suivants pour d'autres comparaisons.

Cependant la différence de performance sur ce jeu de données paraît négligeable face à la différence de complexité entre les algorithmes. Nous n'avons également pas comparé avec précision les temps de calcul qui, sur ce jeu de données, sont courts et donc comparaison peu pertinente.

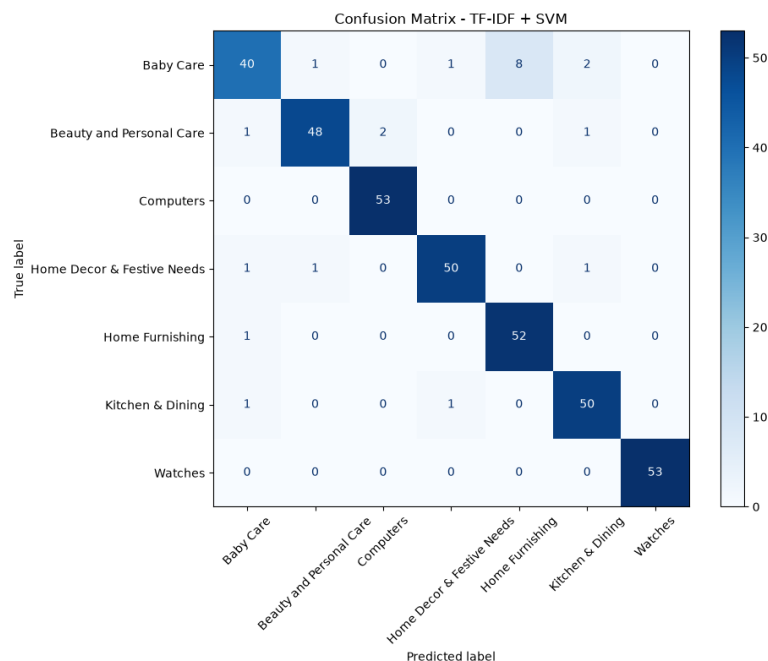
De plus, nos jeux de données et notre problématique sont différents : il est clair que l'évaluation de la pertinence d'un long document par rapport à une requête est autrement plus complexe que notre tâche de classification sur des textes particulièrement courts. Ces résultats ne présentent donc que peu d'intérêt.

L'analyse de la feature importance globale et locale du nouveau modèle

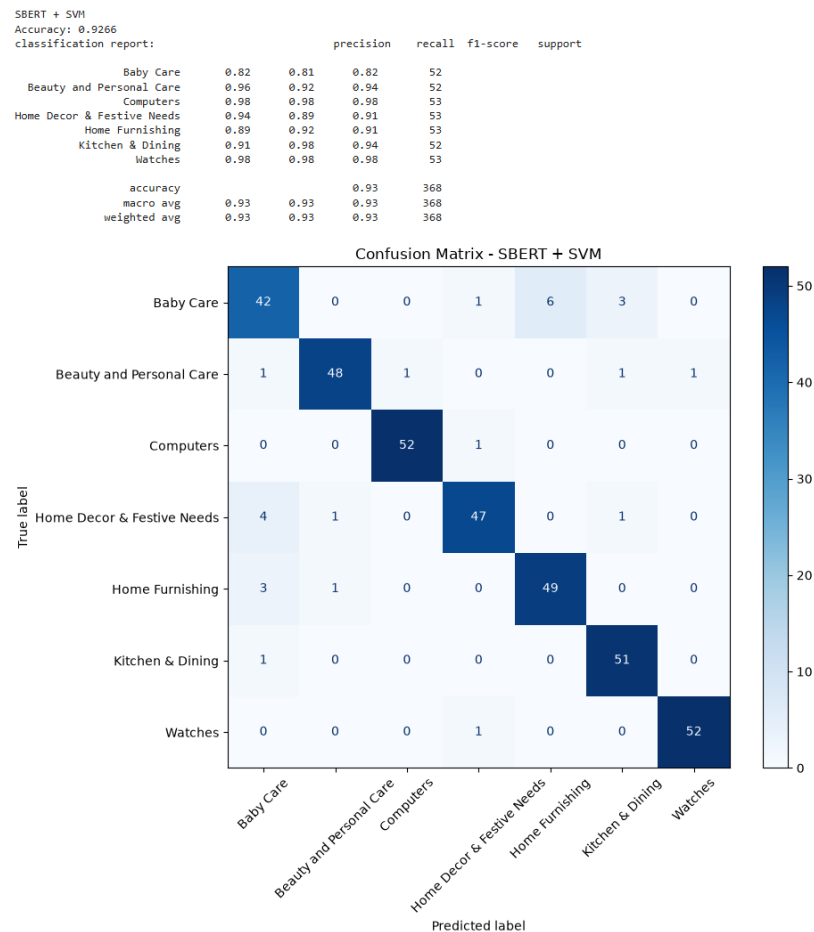
Une étude de la feature importance globale/locale n'a pas beaucoup de sens dans ce contexte. Cependant, une comparaison des algorithmes en enlevant respectivement la composante SBERT et TF-IDF des features est pertinente. Par soucis de temps, de température dans mon appartement, et d'économie de la durée de vie des composants de mon pc, j'ai fait cette comparaison en remplaçant le FCNN par un SVM. TF-IDF seul (en gardant la réduction de dimension) suivi d'un classificateur SVM permet d'obtenir une accuracy de 0,940.

TF-IDF + SVM
Accuracy: 0.9402
classification report:

		precision	recall	f1-score	support
Baby Care	0.91	0.77	0.83	0.80	52
Beauty and Personal Care	0.96	0.92	0.94	0.93	52
Computers	0.96	1.00	0.98	0.99	53
Home Decor & Festive Needs	0.96	0.94	0.95	0.95	53
Home Furnishing	0.87	0.98	0.92	0.95	53
Kitchen & Dining	0.93	0.96	0.94	0.94	52
Watches	1.00	1.00	1.00	1.00	53
accuracy			0.94		368
macro avg	0.94	0.94	0.94	0.94	368
weighted avg	0.94	0.94	0.94	0.94	368



SBERT seul suivi d'un classificateur SVM permet d'obtenir une accuracy de 0,927.



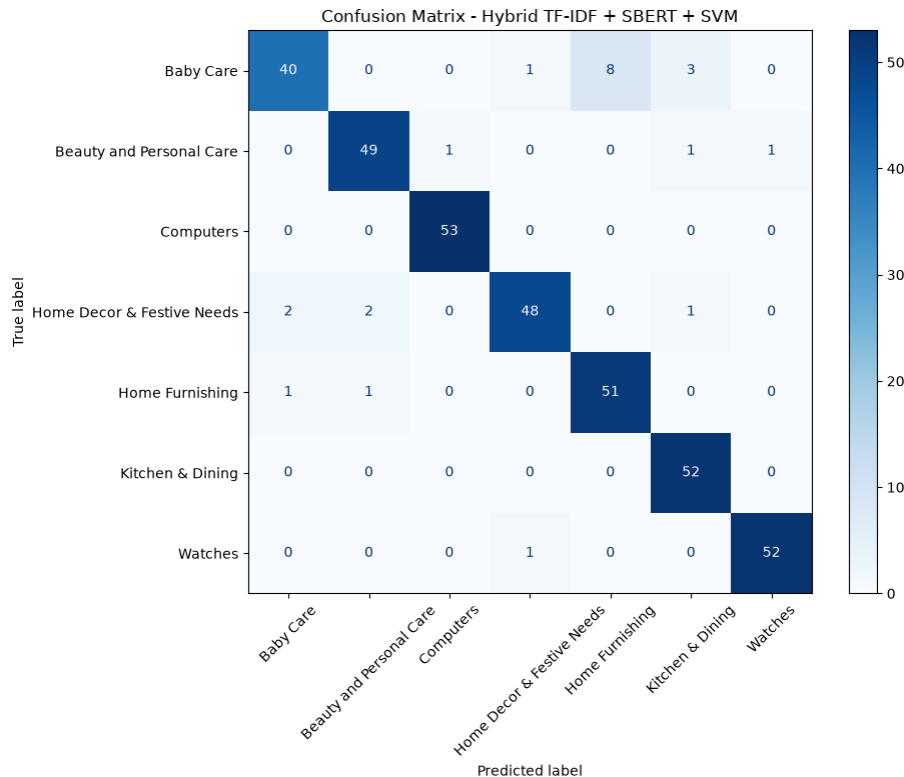
Par rapport à l'accuracy obtenu par SBERT+TF-IDF+SVM (0,938), la différence n'est pas énorme et les erreurs se trouvent toujours entre les classes Baby Care, Kitchen & Dining, et les deux classes de fournitures Home.

Cette conclusion serait certainement valable même avec le FCNN à la place du SVM, et il serait difficile d'avoir des résultats plus significatifs sans un jeu de données bien plus conséquent (ici dataset environ 6x plus petit que celui de l'article).

=== RESULTS ===
Accuracy: 0.9375

Classification Report:

	precision	recall	f1-score	support
Baby Care	0.930	0.769	0.842	52
Beauty and Personal Care	0.942	0.942	0.942	52
Computers	0.981	1.000	0.991	53
Home Decor & Festive Needs	0.960	0.906	0.932	53
Home Furnishing	0.864	0.962	0.911	53
Kitchen & Dining	0.912	1.000	0.954	52
Watches	0.981	0.981	0.981	53
accuracy			0.938	368
macro avg	0.939	0.937	0.936	368
weighted avg	0.939	0.938	0.936	368



Les limites et les améliorations possibles

1) En ce qui concerne les données :

L'expérimentation repose uniquement sur le corpus CISI, qui contient 1 460 documents, 112 requêtes et 3 114 relations de pertinence. Ce corpus est relativement restreint, ancien et spécialisé dans les sciences de l'information. Les résultats obtenus ne permettent donc pas de conclure à une généralisation vers des corpus plus vastes, plus récents ou issus d'autres domaines.

De plus, la constitution d'un jeu équilibré de 3 114 couples pertinents et 3 114 couples non pertinents ne représente pas nécessairement les conditions réelles de recherche d'information, dans lesquelles les documents pertinents sont généralement très minoritaires. Les performances peuvent ainsi être surestimées par rapport à un contexte opérationnel.

2) En ce qui concerne la méthode d'évaluation :

l'évaluation repose principalement sur l'exactitude, qui mesure la proportion de couples correctement classés. Cette métrique est insuffisante pour évaluer un système de recherche d'information, car elle ne renseigne pas directement sur la qualité du classement des documents.

- 3) En ce qui concerne la pipeline en elle-même :
 - Compréhension de la problématique : les représentations SBERT et TF-IDF sont combinées avec des poids fixes, retenus empiriquement comme la meilleure configuration. Cette pondération est identique pour tous les couples requête-document. Elle ne tient donc pas compte du fait que certaines requêtes reposent surtout sur des mots-clés précis, tandis que d'autres nécessitent principalement une compréhension sémantique.
 - Réponse à la problématique : l'objectif opérationnel d'un système de recherche est de classer plusieurs documents les uns par rapport aux autres pour une même requête, ce qui n'est pas fait ici. Une approche plus coûteuse consisterait à conserver deux branches distinctes pour TF-IDF et SBERT, avant de les réunir dans une couche de fusion apprise par le modèle.
 - Interprétabilité : le FCNN apprend des interactions non linéaires entre des centaines de caractéristiques latentes mais ne permet pas d'identifier directement les mots, les passages ou les relations sémantiques responsables de la décision. Remplacer le FCNN par des algorithmes explicables est la seule solution.

En ce qui concerne notre implémentation, les mêmes remarques sont bien sûr valables, mais le problème principal reste la qualité du jeu de données :

- 1) Trop peu de données
- 2) Documents bien trop courts pour que cette technique est un intérêt
- 3) Pipeline/tâche de classification différente : ne fait certainement pas baisser les performances brutes mais améliore le temps de traitement.

Il serait donc intéressant de pouvoir implémenter cette technique avec un jeu de données conséquent et adapté afin de pouvoir tester les améliorations suggérés.